Unit-4 (Image Compression)
Introduction of Compression

Lecture-1

What is image compression?

Image compression is a process applied to a graphics file to minimize its size in bytes without degrading image quality below an acceptable threshold. By reducing the file size, more images can be stored in a given amount of disk or memory space. The image also requires less bandwidth when being transmitted over the internet or downloaded from a webpage, reducing network congestion and speeding up content delivery.

What are the two types of image compression?

The methods used to compress image files typically fall into one of two categories: lossy and lossless. Lossy compression reduces an image file size by permanently removing less critical information, particularly redundant data. Lossy compression can significantly reduce file size, but it can also reduce image quality to the point of distortion, especially if the image is overly compressed. However, quality can be maintained when compression is carefully applied.

One of the challenges with lossy compression is that it's irreversible. Once it has been applied to an image, that image can never be restored to its original state. If lossy compression is applied repeatedly to the same image, it gets increasingly distorted. That said, lossy compression has proved to be a valuable strategy for the web, where a moderate amount of image degradation can often be tolerated.

The most common example of lossy compression is JPEG, an image compression format used extensively on the web and in digital photography. This widely recognized format is supported by numerous tools and applications. Additionally, compression can be applied in degrees, making it possible to use JPEG compression that best strikes a balance between file size and quality.



The difference in quality as more compression is applied to an image.

The other approach to image compression is referred to as lossless. This method applies compression without removing critical data or reducing image quality and results in a compressed image that can be restored to its original state with no degradation or distortion. However, lossless compression doesn't reduce the file size nearly as much as lossy compression, offering little advantage in terms of storage space, network bandwidth or download speeds. Lossless compression is generally used in situations where image quality is more important than disk space or network performance, such as for product images or to showcase artwork.

One of the most common lossless formats is PNG, a widely used format that reduces file size by identifying patterns and compressing those patterns together. Although PNG files are generally larger than JPEG files, websites use them extensively when more image detail is needed, such as for logos, icons, screenshots or images with text. Another familiar lossless format is BMP, a proprietary approach to image compression introduced by Microsoft and used primarily for Microsoft products, particularly Windows computers.

GIF is compression format that falls into the lossless category, although there is some confusion as to whether it is lossy or lossless. GIF images are limited to 256 colors, so converting an image with more colors to GIF results in a loss of quality, which is sometimes attributed to lossy compression. But the compression algorithms used by GIF are lossless. If quality is lost, it's due to issues related to converting the file. Currently, the GIF format is used primarily for simple videos and animations.

Unit-4 (Image Compression)

Lecture-2

CompressionScheme& Coding

Redundancy

What Are The Types Of Image Compression?

Image compression has two prime categories - lossless and lossy image compression. These vary based on the image file resizing process. While the former ensures the image quality remains intact, the latter removes some parts to get a smaller size.

Lossless Compression

Lossless compression refers to a process of resizing the images into a smaller version. This technique does not fiddle with the image quality.

Though it is an excellent method to resize your image files, the outcome may still not be too small. That is because lossless compression does not eliminate any part of the image.

For example, it will convert an image of 15 MB to 10 MB. However, it will still be too large to display on a webpage.

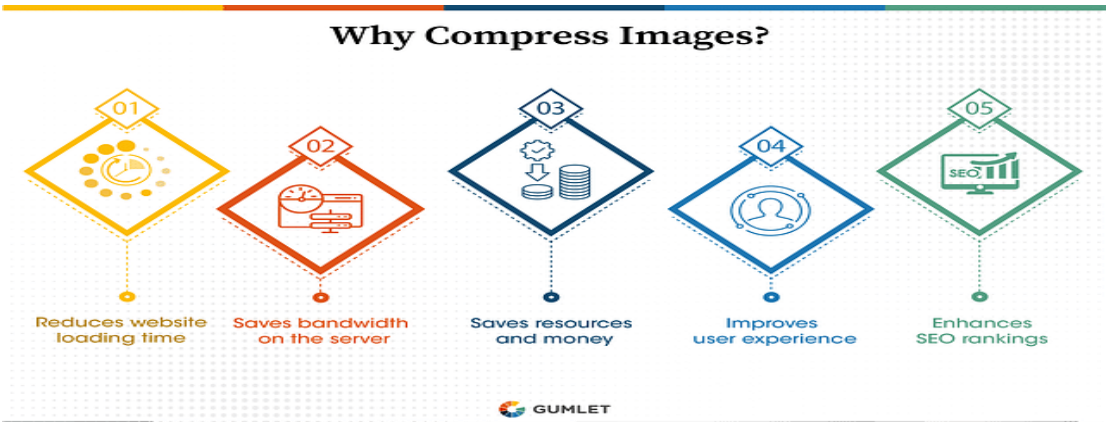
Lossless image compression is particularly useful when compressing text. That is because a small change in the original version can dramatically change the text or data meaning.

Pros

- Image parts remain intact
- Zero loss in image quality
- It is a reversible process

Cons

- The image output is too large
- Decoding is challenging



Lossy Compression
Lossy compression reduces the image size by removing some of the image parts. It eliminates the tags that are not very essential.

If you opt for this method, you can get a significantly smaller version of an image with a minimal quality difference. Additionally, you can enjoy a faster loading speed.

Lossy compression works with a quality parameter to measure the change in quality. In most cases, you have to set this parameter. If it is lower than 90, the images may appear low quality to the human eye.

For example, you can convert an image of 15 MB into 2200 Kb as well as 400 Kb.

That said, Image optimization services like Gumlet do not require you to enter the quality parameter. We use a new technique developed through machine learning - Perceptually Lossless Compression.

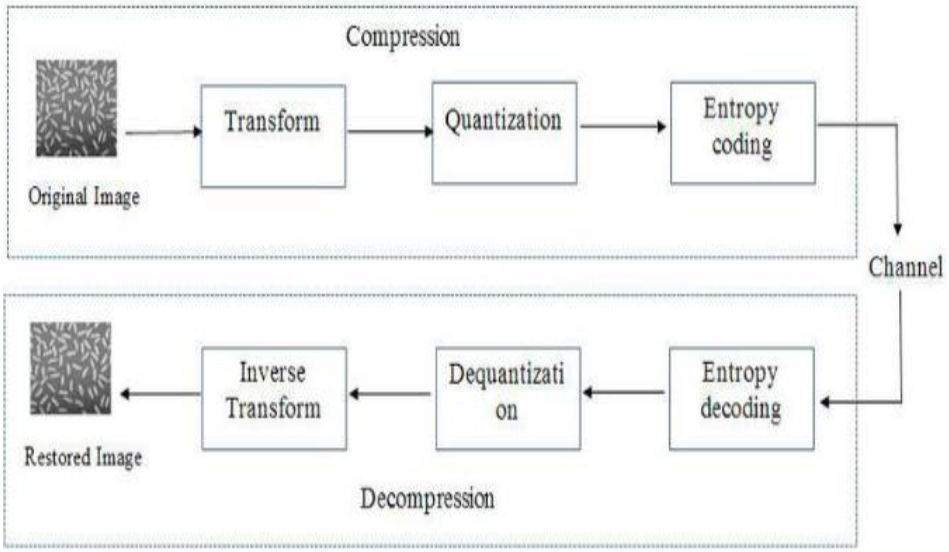
The system automatically identifies the required parameter for lossy image compression.

Pros

- Get a highly reduced image size
- Fast load time
- Ideal option for websites

Cons

- Loses image components
- It is irreversible



Redundancy in Digital Image Processing

Redundancy in Image Processing

To understand Image redundancy or data redundancy in digital image processing lets look at the example. Let assume that 2 people Ramswarup and Suresh said a story. Suresh said the story in fewer words

compare to Ramswarup where Ramswarup took too many words to said the same story. So either Ramswarup said un-relevant information/data which are not the part of the story or maybe he repeated his words more than once.

Redundancy refers to **"storing extra information to represent a quantity of information".** So that is the redundancy of data now apply this concept on digital images we know that computer store the images in pixel values so sometimes image has duplicate pixel values or maybe if we remove some of the pixel values they don't affect the information of an actual image. Data Redundancy is one of the fundamental component of **Data Compression**.

Data Compression

The Data Compression refers to the process of reducing the amount of data required to represent a given quantity of information. We know that one common characteristic followed by all the images is the **neighboring of pixels** and all the pixels are correlated to each other so there is a chance of existing redundant information.

Types of Redundancy in the context of neighboring pixels-: Broadly we have three types of redundancy in images pixels

1. **Spatial Redundancy-:** In spatial redundancy there is a correlation between the neighboring pixel **values**.
2. **Spectral Redundancy-:** In spectral redundancy there is a correlation between different **color planes or spectral bands**.
3. Temporal Redundancy-: In temporal redundancy there is a correlation between **adjacent frames** in the sequence of image.

7

Unit-4 (Image Compression)

Lecture-3

Huffman Coding (GreedyTechnique)

Huffman Codes

- (i) Data can be encoded efficiently using Huffman Codes.
- (ii) It is a widely used and beneficial technique for compressing data.
- (iii) Huffman's greedy algorithm uses a table of the frequencies of occurrences of each character to build up an optimal way of representing each character as a binary string.

Suppose we have 10^5 characters in a data file. Normal Storage: 8 bits per character (ASCII) - 8 x 10^5 bits in a file. But we want to compress the file and save it compactly. Suppose only six characters appear in the file:

	a	b	c	d	e	f	Total
Frequency	45	13	12	16	9	5	100

How can we represent the data in a Compact way?

(i) Fixed length Code: Each letter represented by an equal number of bits. With a fixed length code, at least 3 bits per character:

For example:

Play Video	
a	000
b	001
c	010
d	011
e	100
f	101

For a file with 10^5 characters, we need 3 x 10^5 bits.

(ii) A variable-length code: It can do considerably better than a fixed-length code, by giving many characters short code words and infrequent character long codewords.

For example:

a 0

b 101

c 100

d 111

e 1101

f 1100

Number of bits = (45 x 1 + 13 x 3 + 12 x 3 + 16 x 3 + 9 x 4 + 5 x 4) x 1000

= 2.24 x 10^5bits

Thus, 224,000 bits to represent the file, a saving of approximately 25%.This is an optimal character code for this file.

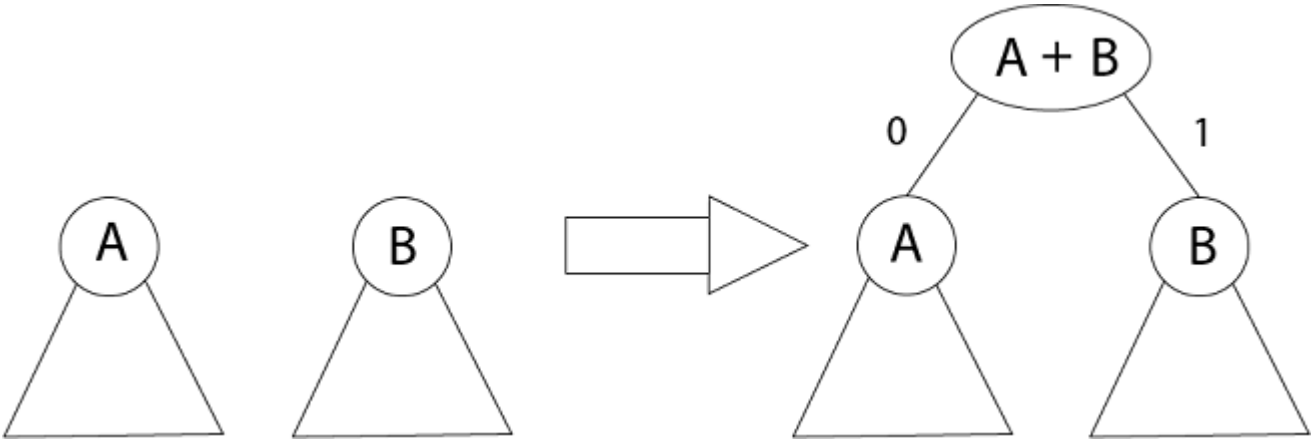
Prefix Codes:

The prefixes of an encoding of one character must not be equal to complete encoding of another character, e.g., 1100 and 11001 are not valid codes because 1100 is a prefix of some other code word is called prefix codes.

Prefix codes are desirable because they clarify encoding and decoding. Encoding is always simple for any binary character code; we concatenate the code words describing each character of the file. Decoding is also quite comfortable with a prefix code. Since no codeword is a prefix of any other, the codeword that starts with an encoded data is unambiguous.

Greedy Algorithm for constructing a Huffman Code:

Huffman invented a greedy algorithm that creates an optimal prefix code called a Huffman Code.



The algorithm builds the tree T analogous to the optimal code in a bottom-up manner. It starts with a set of |C| leaves (C is the number of characters) and performs |C| - 1 'merging' operations to create the final tree. In the Huffman algorithm 'n' denotes the quantity of a set of characters, z indicates the parent node, and x & y are the left & right child of z respectively.

Unit-4 (Image Compression)

Lecture-4

Arithmetic Coding

It is a lossless compression algorithm with no loss of data aftercompression.

Arithmetic coding is a popular compression algorithm after Huffman coding and it is particularly useful for a relatively small and skewed alphabet. We only discuss the static approach here. In theory, an arithmetic coding algorithm encodes an entire file as a sequence of symbols into a single decimal number. The input symbols are processed one at each iteration. The initial interval [0, 1) (or [0, 1]) is successively divided into subintervals on each iteration according to the probability distribution. The subinterval that corresponds to the input symbol is selected for next iteration. The interval derived at the end of this division process is used to decide the codeword for the entire sequence of symbols. Unfortunately, implementation often encounters difficulties due to the constraints of computer precision.

Applications

Arithmetic coding is used in a variety of lossless and lossy compression applications. It is a part of many international standards. In the area of multimedia there are a few principal organizations that develop standards. The International Standards Organization (ISO) and the International Electrotechnical Commission (IEC) are industry groups that work on multimedia standards, while the International Telecommunications Union (ITU), which is part of the United Nations, works on multimedia standards on behalf of the member states of the United Nations. Quite often these institutions work together to create international standards. In later chapters, we will be looking at a number of these standards; and we will see how arithmetic coding is used in image compression, audio compression, and video compression standards.

For now, let us look at the lossless compression example from the previous chapter.

In Tables 4.7 and 4.8, we show the results of using adaptive arithmetic coding to encode the same test images that were previously encoded using Huffman coding. We have included the compression ratios obtained using Huffman code from the previous chapter for comparison. Comparing these values to those obtained in the previous chapter, we can see very little change. The reason is that because the alphabet size for the images is quite large, the value of σ_{max} is quite small; and the Huffman coder performs very close to the entropy.

Arithmetic coding:

Unlike the variable-length codes described previously, arithmetic coding generates nonblock codes. In arithmetic coding, which can be traced to the work of Elias, a one-to-one correspondence between source symbols and code words does not exist. Instead, an entire sequence of source symbols (or message) is assigned a single arithmetic code word. The code word itself defines an interval of real numbers between 0 and 1. As the number of symbols in the message increases, the interval used to represent it becomes smaller and the number of information units (say, bits) required to represent the interval becomes larger. Each symbol of the message reduces the size of the interval in accordance with its probability of occurrence. Because the technique does not require, as does Huffman's approach, that each source symbol translate into an integral number of code symbols (that is, that the symbols be coded one at a time), it achieves (but only in theory) the bound established by the noiseless coding theorem.

. Here, a five-symbol sequence or message, $a_1a_2a_3a_3a_4$, from a four-symbol source is coded. At the start of the coding process, the message is assumed to occupy the entire half-open interval [0, 1). As Table 5.2 shows, this interval is initially subdivided into four regions based on the probabilities of each source symbol. Symbol ax, for example, is associated with subinterval [0, 0.2). Because it is the first symbol of the message being coded, the message interval is initially narrowed to [0, 0.2). Thus in Fig. 5.1 [0, 0.2) is expanded to the full height of the figure and its end points labeled by the values of the narrowed range. The narrowed range is then subdivided in accordance with the original source symbol probabilities and the process continues with the next message symbol.

In this manner, symbol a_2 narrows the subinterval to [0.04, 0.08), a_3 further narrows it to [0.056, 0.072), and so on. The final message symbol, which must be reserved as a special end-of-message indicator, narrows the range to [0.06752, 0.0688). Of course, any number within this subinterval—for example, 0.068—can be used to represent the message.

In the arithmetically coded message of Fig. 5.1, three decimal digits are used to represent the five-symbol message. This translates into 3/5 or 0.6 decimal digits per source symbol and compares favorably with the entropy of the source, which is 0.58 decimal digits or 10- ary units/symbol. As the length of the sequence being coded increases, the resulting arithmetic code approaches the bound established by the noiseless coding theorem.

In practice, two factors cause coding performance to fall short of the bound: (1) the addition of the end-of-message indicator that is needed to separate one message from an- other; and (2) the use of finite precision arithmetic. Practical implementations of arithmetic coding address the latter problem by introducing a scaling strategy and a rounding strategy (Langdon and Rissanen [1981]). The scaling strategy renormalizes each subinterval to the [0, 1) range before subdividing it in accordance with the symbol probabilities. The rounding strategy guarantees that the truncations associated with finite precision arithmetic do not prevent the coding subintervals from being represented accurately

Unit-4 (Image Compression)

Lecture-5

JPEGCompression

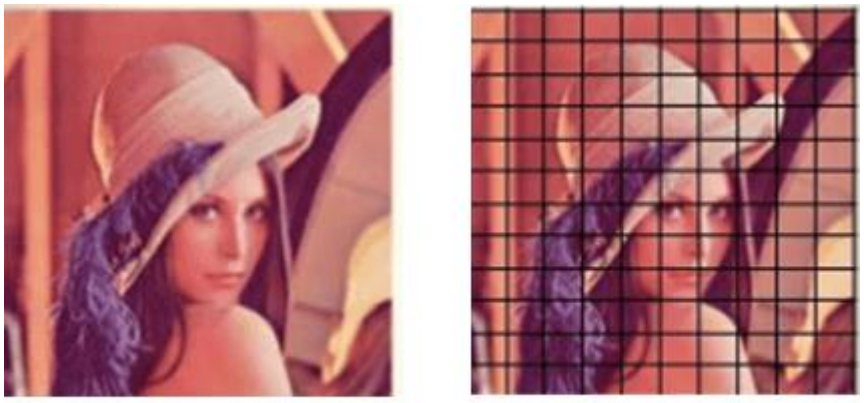
JPEG-

Joint Photographic Experts Group.

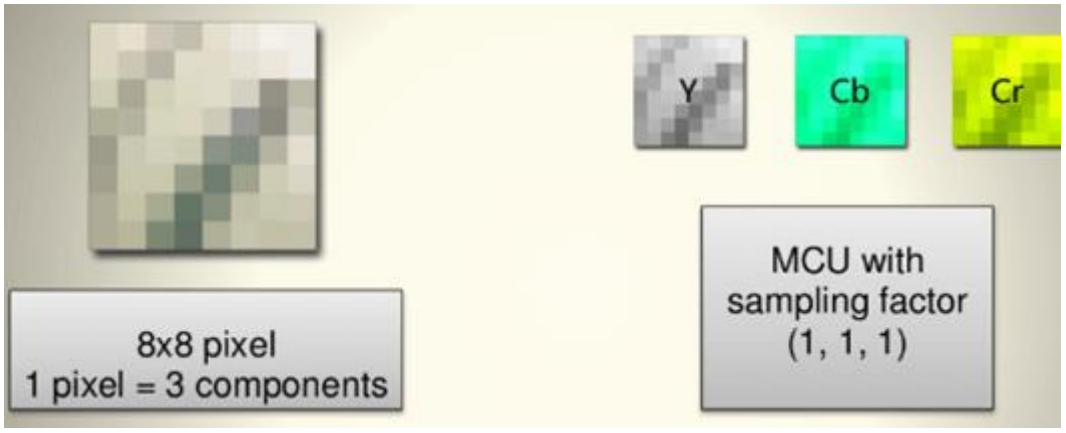
DCT (Discrete Cosine Transform) method for coding transformation. It allows a tradeoff between storage size and the degree of compression can be adjusted.

Following are the steps of JPEG Image Compression-

Step 1: The input image is divided into a small block which is having 8x8 dimensions. This dimension is sum up to 64 units. Each unit of the image is called pixel.



Step 2: JPEG uses [Y,Cb,Cr] model instead of using the [R,G,B] model. So in the 2nd step, RGB is converted into YCbCr.



Step 3: After the conversion of colors, it is forwarded to DCT. DCT uses a cosine function and does not use complex numbers. It converts information's which are in a block of pixels from the spatial domain to the frequency domain.

DCT Formula

$$F(w) = \frac{a(u)}{2} \sum_{n=0}^{N-1} f(n) \cos \frac{(2n+1)w\pi}{16}$$

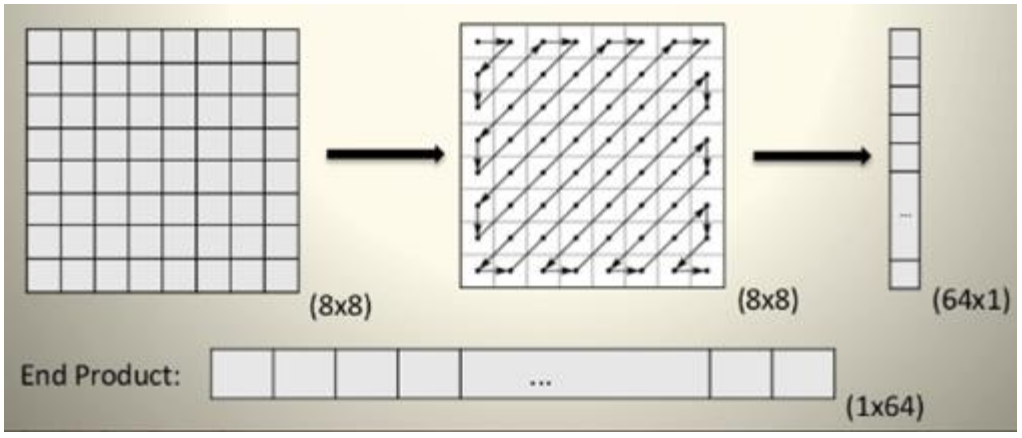
Step 4: Humans are unable to see important aspects of the image because they are having high frequencies. The matrix after DCT conversion can only preserve values at the lowest frequency that to in certain point. Quantization is used to reduce the number of bits per sample.

There are two types of Quantization:

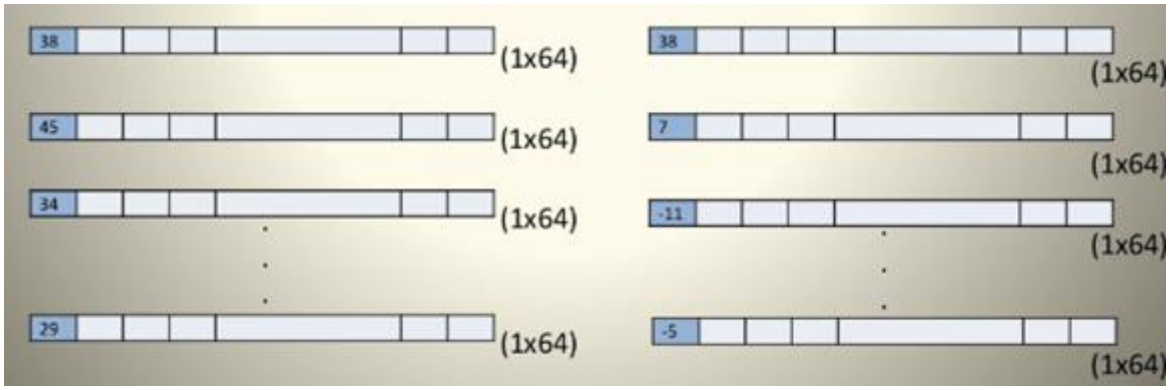
1. Uniform Quantization
2. Non-Uniform Quantization

Standard Formula: $\widehat{F}(\mathbf{u},v) = round\left(\frac{\mathbf{F(u,v)}}{\mathbf{Q(u,v)}}\right)$

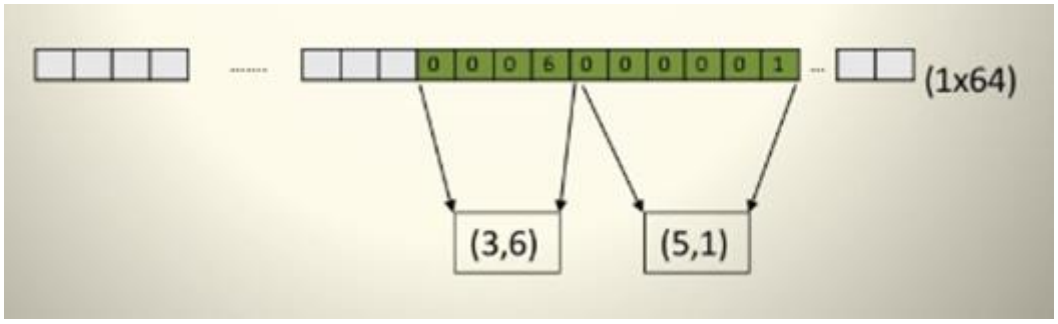
Step 5: The zigzag scan is used to map the 8x8 matrix to a 1x64 vector. Zigzag scanning is used to group low-frequency coefficients to the top level of the vector and the high coefficient to the bottom. To remove the large number of zero in the quantized matrix, the zigzag matrix is used.



Step 6: Next step is vectoring, the different pulse code modulation (DPCM) is applied to the DC component. DC components are large and vary but they are usually close to the previous value. DPCM encodes the difference between the current block and the previous block.



Step 7: In this step, **Run Length Encoding (RLE)** is applied to AC components. This is done because AC components have a lot of zeros in it. It encodes in pair of (skip, value) in which skip is non zero value and value is the actual coded value of the non zero components.



Step 8: In this step, DC components are coded into Huffman.

SIZE	Value	Code
0	0	---
1	-1,1	0,1
2	-3, -2, 2,3	00,01,10,11
3	-7,..., -4, 4,..., 7	000,..., 011, 100,...111
4	-15,..., -8, 8,..., 15	0000,..., 0111, 1000,..., 1111
.		.
.		.
11	-2047,..., -1024, 1024,... 2047	...

Huffman table for DC components value field

SIZE	Code Length	Code
0	2	00
1	3	010
2	3	011
3	3	100
4	3	101
5	3	110
6	4	1110
7	5	11110
8	6	111110
9	7	1111110
10	8	11111110
11	9	111111110

Huffman table for DC components Size field